Artificial Intelligence

Exam Revision Notes - Revised Edition

Chapters 1-8 corrected and re-weighted, plus Chapters 10, 14 and 15

Based on George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th edition

What changed from the first version

- **Errors fixed:** financial-advisor rules 5, 7, 8; the sorting production-system trace; the distributive law; the monotone-heuristic formula; the binomial formula; the naive Bayes product; and the typos (notable→ontable, onb(a)→on(b,a), min→nim, plies, S0, 1.26^N, 9!, Kolodner, Coderack, HEARSAY).
- **Re-weighted** toward the pattern of Dr V.R. Baloyi's 2024-2026 SCOA032 papers: more on Chapters 3, 4, 6 and 7 and the expert-system architecture; Chapter 5 cut to a short summary; Livingstone, bridging faults and teleo-reactive trees removed; Chapter 1 history trimmed.
- **Added:** a full Chapter 6 expansion, Computer Vision, responsive vs proactive agents, bidirectional search, and an appendix on Chapters 10 (machine learning), 14 (automated reasoning) and 15 (natural language).
- **Not included:** the textbook figures (trees, graphs, Venn diagrams) from the first version. Redraw the ones you need from the book.

Read this first

- The exam weightings and 'marks in paper X' claims come from an earlier analysis of the past papers that was pasted into this chat. I could not check the papers themselves, so treat the weightings as a guide and confirm against your own copies of the papers and the module outline.
- Colour key: blue boxes = definitions, grey = algorithms, green = notes, red = exam focus.

Contents

- 1 AI: History and Applications (trimmed, plus Computer Vision and agents)
- 2 The Predicate Calculus (corrected)
- 3 State Space Search (traces, goal vs data driven, bidirectional)
- 4 Heuristic Search (alpha-beta, A*, corrected formulas)
- 5 Stochastic Methods (short self-study summary)
- 6 Control Algorithms and Production Systems (expanded)
- 7 Knowledge Representation (scripts, frames, conceptual graphs)
- 8 Strong Method Problem Solving (expert systems, STRIPS)
- Appendix A: Ch 10 Machine Learning, Ch 14 Automated Reasoning, Ch 15 Natural Language
- Cheat sheet

Chapter 1 - AI: History and Applications

1.1 Defining AI

Definition (Luger)

- AI is the branch of computer science concerned with the **automation of intelligent behaviour**. It rests on data structures (knowledge representation), algorithms (applying the knowledge) and languages/programming techniques.
- Weakness: intelligence itself is not well defined, so evaluating a program is hard.
- Working definition: AI is the collection of problems and methods studied by AI researchers.

1.2 History in brief (one-liners)

Person / event	Why it matters
Aristotle	Matter vs form; logic and the syllogism (basis of symbolic reasoning).
Descartes	Mind-body dualism: mental processes can be studied on their own.
Rationalists vs empiricists	Rationalists: reason reconstructs the world (logic, planners, Physical Symbol System Hypothesis). Empiricists: knowledge comes from experience and association (semantic networks, connectionism).
Boole, Frege, Russell	Boolean algebra; predicate calculus; mathematics as a formal system, so reasoning can be mechanised.
Euler	Graph theory (Königsberg bridges), the basis of state space search.
Turing (1950)	Imitation game: an objective test of intelligence by behaviour alone.
Early programs	Logic Theorist, GPS (reasoning); DENDRAL, MYCIN, XCON (expert systems); SHRDLU (language in a micro-world).

1.3 The Turing test

- An interrogator talks by text to a human and a machine; if they cannot tell which is which, the machine is called intelligent.
- **Advantages:** objective notion of intelligence; avoids unanswerable questions about internal processes and consciousness; removes bias toward living organisms.
- **Criticisms:** tests only symbolic problem solving (no perception or dexterity); forces machines into a human mould; Lovelace's objection (computers only do what they are told); informality of behaviour.

1.4 Agents

Five themes of the agent-oriented view

- **Autonomous:** each agent has its own responsibilities. **Situated:** sensitive to its own environment, not the whole domain. **Interactional:** agents cooperate like a society. **Structured:** agents coordinate, so the solution is collective. **Emergent:** the result is greater than the sum of the parts.

Responsive (reactive) vs proactive agents

- **Responsive:** perceives its environment and responds in a timely way to changes in it.
- **Proactive:** does not simply react; it shows goal-directed behaviour and takes the initiative to achieve its goals.
- Luger's 'flexible' criterion for a multi-agent system combines both (credit agent that tries another agency = reactive; planning ahead = proactive).

1.5 Application areas

Area	Key points
Game playing	Well-defined rules; huge spaces need heuristics (useful but fallible).
Theorem proving	Oldest branch; logic is formal; early provers drowned in irrelevant theorems.
Expert systems	Power from domain knowledge obtained by a knowledge engineer: DENDRAL, MYCIN, XCON.
Natural language	Needs large background knowledge, not just parsing.
Planning and robotics	Sequence of actions to reach a goal; robots must react to a changing world.
Machine learning	Expert systems re-solve the same problem every time; learning programs improve with experience.
Neural nets, genetic algorithms	Robust to noise, parallel; GAs use crossover and mutation.

Computer vision (medical / radiology scenario)

- **What to recommend:** an image-processing and recognition system that analyses scans (X-ray, CT) and flags suspicious regions for the radiologist (decision support, not replacement).
- **Architecture elements:** (1) image acquisition/sensing; (2) pre-processing (noise reduction, contrast); (3) segmentation (separate regions of interest); (4) feature extraction; (5) recognition/classification; (6) a domain knowledge base; (7) output and explanation to the clinician.
- **Current deficits:** sensitive to lighting, artefacts and equipment differences (brittle generalisation); lacks deep contextual and commonsense understanding; needs large labelled data; cannot explain well; errors have serious consequences.

Five deficiencies of expert systems

1. Difficulty capturing deep knowledge (MYCIN once asked if a male patient was pregnant).
2. Lack of robustness and flexibility; cannot go back to first principles.
3. Inability to give deep explanations (only the steps taken).
4. Difficulty of verification.
5. Little learning from experience; knowledge is handcrafted.

Luger's 8 common features of AI

- Computers used for reasoning, pattern recognition, learning or inference.
- Problems that do not respond to algorithmic solutions (so heuristic search).
- Inexact, missing or poorly defined information.
- Reasoning about significant qualitative features.
- Semantic meaning as well as syntactic form.
- Answers that are neither exact nor optimal but sufficient.
- Large amounts of domain-specific knowledge.
- Meta-level knowledge for more sophisticated control.

Exam focus (Dr Baloyi / SCOA032 style)

- Define AI and say why it is hard to define; explain the Turing test with advantages and criticisms.
- Give the five agent themes and contrast responsive with proactive agents.
- Describe a computer vision system for a given scenario: type, architecture, deficits.
- List expert-system deficiencies.

Chapter 2 - The Predicate Calculus

Predicate calculus is AI's representation language: a well-defined syntax, a formal semantics, and sound inference rules. Search, expert systems and Prolog build on it.

2.1 Propositional calculus

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ≡ Q	¬P ∨ Q
T	T	F	T	T	T	T	T
T	F	F	F	T	F	F	F
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	T

The columns **P → Q** and **¬P ∨ Q** are identical, which proves material implication. Implication is false only when P is true and Q is false. To prove two expressions equivalent, show their truth-table columns are identical.

Name	Identity
Double negation	¬¬P ≡ P
Material implication	(P → Q) ≡ (¬P ∨ Q)
Contrapositive	(P → Q) ≡ (¬Q → ¬P)
De Morgan	¬(P ∨ Q) ≡ (¬P ∧ ¬Q); ¬(P ∧ Q) ≡ (¬P ∨ ¬Q)
Commutative	(P ∧ Q) ≡ (Q ∧ P); (P ∨ Q) ≡ (Q ∨ P)
Associative	((P ∧ Q) ∧ R) ≡ (P ∧ (Q ∧ R)); ((P ∨ Q) ∨ R) ≡ (P ∨ (Q ∨ R))
Distributive (corrected)	P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R) and P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R)

Propositional calculus is **decidable** (n symbols give 2^n interpretations). Predicate calculus is **not decidable** (domains can be infinite).

2.2 Predicate calculus syntax

- Symbols start with a letter; letters, digits and underscore only. **Constants** start lowercase (george), **variables** uppercase (X), **functions** and **predicates** lowercase with an arity.
- **Term** = constant, variable or function expression. **Atomic sentence** = predicate of arity n followed by n terms: likes(george, kate). plus(two, three) is a function expression, not a sentence.
- Well-formedness and truth are independent: equal(plus(2,3), seven) is well formed but false.
- Sentence rules: atomic sentences; ¬S; S_1 ∧ S_2; ∨; →; ≡; ∀X S; ∃X S.
- Free variable: not in the scope of a quantifier. Ground expression: no variables. Closed expression: all variables quantified.

English	Predicate calculus
All basketball players are tall.	∀X (basketball_player(X) → tall(X))
Some people like anchovies.	∃X (person(X) ∧ likes(X, anchovies))
Nobody likes taxes.	¬∃X likes(X, taxes)
If it does not rain on Monday, Tom will go to the mountains.	¬weather(rain, monday) → go(tom, mountains)
Emma is a Doberman pinscher and a good dog.	gooddog(emma) ∧ isa(emma, doberman)

Trap: 'all' uses → ; 'some' uses ∧. Quantifier equivalences: ¬∃X p(X) ≡ ∀X ¬p(X); ¬∀X p(X) ≡ ∃X ¬p(X); ∀ distributes over ∧; ∃ distributes over ∨.

2.3 Inference, soundness and unification

Term	Definition
Satisfiable / unsatisfiable	Some interpretation makes it true / none does (∃X (p(X) ∧ ¬p(X))).
Valid	True under every interpretation (∀X (p(X) ∨ ¬p(X))).
Model	An interpretation that satisfies the sentence for all variable assignments.
Sound rule	Everything it produces logically follows from the premises.
Complete rule	It can produce everything that logically follows.
Modus ponens / tollens	P, P → Q ⊢ Q / P → Q, ¬Q ⊢ ¬P
Universal instantiation	∀X p(X) ⊢ p(a) for a ground term a.

Unification

- Finds substitutions that make two expressions identical. A constant cannot be replaced; two different constants do not unify.

- A variable may be bound to a constant, a term or another variable. **Occurs check:** X cannot unify with f(X).
- Apply substitutions consistently; composition is associative, not commutative. Return the **most general unifier** (mgu).
- Skolemization removes ∃: ∀X ∃Y mother(X,Y) becomes ∀X mother(X, f(X)).

Expression 1	Expression 2	Unifier
foo(X, a, goo(Y))	foo(fred, a, goo(Z))	{fred/X, Z/Y}
foo(X, a, goo(Y))	foo(Z, a, goo(moo(Z)))	{Z/X, moo(Z)/Y}
p(X)	p(Y)	{Z/X, Z/Y} (most general)
X	f(X)	FAIL (occurs check)
parents(X, father(X), mother(bill))	parents(bill, father(bill), Y)	{bill/X, mother(bill)/Y}

2.4 The financial advisor (corrected)

minsavings(X) = 5000 × X; minincome(X) = 15000 + 4000 × X (X = dependents); income must also be steady.

```
1. savings_account(inadequate) → investment(savings)
2. savings_account(adequate) ∧ income(adequate) → investment(stocks)
3. savings_account(adequate) ∧ income(inadequate) → investment(combination)
4. ∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ greater(X, minsavings(Y))) → savings_account(adequate)
5. ∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minsavings(Y))) → savings_account(inadequate)
6. ∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ greater(X, minincome(Y))) → income(adequate)
7. ∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minincome(Y))) → income(inadequate)
8. ∀X earnings(X, unsteady) → income(inadequate)
```

Rules 1-3 give the advice; rules 4-8 decide adequacy. Rules 5 and 7 need **¬greater**; rule 8 concludes **inadequate**. Worked case: 2 dependents, $20,000 saved, $30,000 steady income: minsavings(2) = 10,000 so savings adequate; minincome(2) = 23,000 so income adequate; advice = **stocks**.

Exam focus (Dr Baloyi / SCOA032 style)

- Translate English to predicate calculus (∀ with →, ∃ with ∧); decide if an expression is well formed.
- Prove equivalence or soundness with truth tables.
- Unify step by step; spot the occurs check; give the mgu.
- Define sound, complete, valid, satisfiable; apply modus ponens and universal instantiation.
- Reproduce the financial advisor rules and run them.

Chapter 3 - Structures and Strategies for State Space Search

3.1 Definitions

Term	Definition
Graph / directed	Set of nodes N and arcs (ordered pairs of nodes).
Tree	Unique path between every pair of nodes; each node has one parent.
DAG	Directed acyclic graph (tic-tac-toe space).
Cycle	A path that contains a node more than once.
Rooted graph / leaf	Unique root with no parent / node with no children.
FSM	Triple (S, I, F): states, inputs, transition function. A finite state acceptor adds a start state s_0 and accept states.
State space	Four-tuple [N, A, S, GD]: nodes, arcs, start state(s), goal state(s). Goal described by a property of a state (type 1) or of the whole path (type 2, e.g. travelling salesperson).

- Euler: a walk crossing every bridge once exists only if the graph has exactly 0 or 2 nodes of odd degree; Königsberg has four, so it is impossible.
- Tic-tac-toe is a DAG with 9! = 362,880 paths; chess about 10^{120}; checkers about 10^{40}. TSP exhaustive search is (N-1)!; branch and bound is still exponential (about 1.26^N); nearest neighbour is fast but fallible.
- The 8-puzzle graph has cycles, so general graph search must detect loops.

3.2 Data-driven vs goal-driven search

	Data-driven (forward chaining)	Goal-driven (backward chaining)
Start / method	Start from given facts; apply rules to produce new facts until the goal is reached.	Start from the goal or hypothesis; find rules whose conclusion matches; their premises become subgoals.
Use when (three factors)	1. All or most of the data are given in the problem statement (interpretation). 2. There is a large number of potential goals but only a few ways to use the facts. 3. It is difficult to form a goal or hypothesis (DENDRAL, PROSPECTOR).	1. A goal or hypothesis is given or easily formulated (theorem proving, diagnosis). 2. There are many rules matching the facts, giving a high branching factor going forward. 3. Problem data are not given and must be acquired by the solver (a doctor orders only the tests needed).

Both search the same graph but visit different states. Jefferson example: searching back from 'I' examines about 2^{10} ancestors; forward from Jefferson about 3^{10} descendants, so backward is better.

Bidirectional (combined) search

- Search forward from the start and backward from the goal at once, hoping the two frontiers meet.
- Compare three cases: states examined by **forward search only** (large fan-out cone from the start); by **backward search only** (cone from the goal); and by the **combination**, where each frontier stays small and meets in the middle, examining far fewer states when the branching factor is constant and the search exhaustive.
- Caution: with heuristic search the two frontiers may **miss** each other; in a production system you can run forward until the state count grows, then switch to goal-directed.

3.3 Backtracking

Backtrack data structures

- **CS** (current state): the state being examined. **SL** (state list): states on the current path; returned as the solution if the goal is found.
- **NSL** (new state list): nodes awaiting evaluation. **DE** (dead ends): states whose descendants contain no goal; never revisited.
- Each new child is tested for membership in SL, NSL and DE to prevent loops. At a dead end, add CS to DE, pop SL and NSL and continue from the next node on NSL.

Trace for the textbook tree (A has children B, C, D; B has E, F; E has H, I; F has J; C has G):

Iter	CS	SL	NSL	DE
0	A	[A]	[A]	[ ]
1	B	[B A]	[B C D A]	[ ]
2	E	[E B A]	[E F B C D A]	[ ]
3	H	[H E B A]	[H I E F B C D A]	[ ]
4	I	[I E B A]	[I E F B C D A]	[H]

Iter	CS	SL	NSL	DE
5	F	[F B A]	[F B C D A]	[E I H]
6	J	[J F B A]	[J F B C D A]	[E I H]
7	C	[C A]	[C D A]	[B F J E I H]
8	G	[G C A]	[G C D A]	[B F J E I H]

Exam technique: on a large tree (for example 19 nodes) write one row per iteration; children go on the *front* of NSL, and when a node has no unvisited children move it to DE.

3.4 Breadth-first, depth-first, iterative deepening

	Breadth-first	Depth-first
open is a	queue (add children on the right)	stack (add children on the left)
Shortest path?	Yes	No
Space	Exponential B^n	Linear in path length
Risk	Memory explosion	Lost in a deep or infinite path

```
function breadth_first_search;
  open := [Start]; closed := []
  while open ≠ [] do
    remove leftmost state from open, call it X;
    if X is a goal then return SUCCESS
    else generate children of X; put X on closed;
         discard children already on open or closed;   % loop check
         put remaining children on RIGHT end of open     % queue
  return FAIL
```

Iterative deepening (Korf): depth-first with bound 1, 2, 3, … It finds the shortest path with linear space (B × n); time is still $O(B^n)$. For the textbook graph: DFS visits A B E K S L T F M C G N H O P U D I Q J R; BFS visits A B C D E F G H I J K L M N O P Q R S T U.

3.5 And/or graphs

- **And node** (q ∧ r → p): all children must be solved (problem decomposition), joined by a curved arc. **Or node** (q ∨ r → p): any child suffices (a choice of strategy).
- Facts a, b, c; rules a∧b→d, a∧c→e, b∧d→f, f→g, a∧e→h. Goal-driven proof of h: prove a and e; e needs c and a, both given. Data-driven: from a, b, c derive d and e, then f and h.
- Fred and Sam: rule 7 unifies with {fred/X, X/Z} to give subgoals gooddog(fred) ∧ master(fred,Y) ∧ location(Y,X); the answer is location(fred, museum).
- Grammar: left-hand sides are and nodes; several rules with the same left side form or nodes; parsing is data-driven, generation goal-driven.

Exam focus (Dr Baloyi / SCOA032 style)

- Define graph, tree, DAG, state space four-tuple, goal types 1 and 2.
- Trace backtrack (CS, SL, NSL, DE) iteration by iteration on a large tree.
- Choose data-driven or goal-driven for a scenario and give the three factors.
- Explain bidirectional search and compare states examined; trace BFS/DFS with open and closed; compare BFS, DFS and iterative deepening.

Chapter 4 - Heuristic Search

Heuristic

- From the Greek *eurisco*, 'I discover': a rule for choosing the branches most likely to lead to an acceptable solution. It is an informed guess and is **fallible**: it can give a suboptimal solution or none.
- **Two situations that need heuristics:** (1) the problem has no exact solution because of ambiguity in the problem statement or data (medical diagnosis, vision); (2) an exact solution exists but the computational cost of finding it is prohibitive (combinatorial explosion, e.g. chess).
- Why they fail: they use limited information (a bad distributor defeats 'check the battery first'); a local maximum looks best but is not the goal.

4.1 Hill climbing, dynamic programming, best-first search

- **Hill climbing:** expand the current state, move to the best child; no siblings or parent kept, so it cannot recover from a **local maximum**.
- **Dynamic programming** (Bellman): store and reuse solutions to subproblems in an array. Minimum edit distance: insertion 1, deletion 1, replacement 2, match 0. Cell = min(above + 1, left + 1, diagonal + 0 or 2). Trace back from the last cell to read off the edits.
- **Best-first search:** like BFS/DFS but **open is a priority queue ordered by heuristic merit**; it recovers from bad choices by going back to the next-best state on open.

```
function best_first_search;
  open := [Start]; closed := [];
  while open ≠ [] do
    remove leftmost state from open, call it X;
    if X = goal then return the path from Start to X
    else generate children of X;
      for each child:
        not on open or closed:  assign heuristic value; add to open
        already on open:  if reached by a shorter path, give it the shorter path
        already on closed: if reached by a shorter path, remove from closed; add to open
      put X on closed;
      re-order open by heuristic merit (best leftmost)
  return FAIL
```

Trace layout: write open and closed after every step, for example open = [A5] → [B4, C4, D6] → [C4, E5, F5, D6] → [H3, G4, E5, F5, D6] → … until the goal is at the front.

4.2 A* and admissibility

Definitions

- **f(n) = g(n) + h(n):** g = actual path length from the start to n; h = heuristic estimate to a goal. g gives search a breadth-first flavour.
- **Admissible:** always terminates with the optimal path when one exists. **A*:** algorithm A with $h(n) \le h^*(n)$ for all n (h never overestimates). All A* algorithms are admissible; breadth-first is A* with h = 0.
- **Monotone (consistent):** $h(n_i) - h(n_j) \le \text{cost}(n_i, n_j)$ for every descendant n_j of n_i, and h(goal) = 0. (The first version's formula h(n) - h(n) ≤ cost(n, n) was wrong.)
- **More informed:** if $h_1(n) \le h_2(n)$ for all n then h_2 is more informed and expands a subset of the states. For the 8-puzzle: 0 ≤ tiles out of place ≤ sum of distances ≤ h*.

Monotone ⇒ admissible: along a path $s_1 \dots s_g$ add the inequalities $h(s_i) - h(s_{i+1}) \le \text{cost}(s_i, s_{i+1})$. The left sides telescope to $h(s_1) - h(s_g) = h(s_1)$, so $h(s_1)$ ≤ the path cost. The converse does not hold in general.

8-puzzle heuristic	Admissible?
Tiles out of place	Yes
Sum of distances to goal squares	Yes (more informed)
2 × direct tile reversals	Poor alone (0 on many states); combine with sum of distances

4.3 Minimax and alpha-beta pruning

- MAX takes the **maximum** of its children, MIN the **minimum**. A 4-ply tree alternates MAX, MIN, MAX, MIN from the root down to the leaves, so evaluate from the leaves upward.
- The **winning path** is found by following, from the root, the child whose value was backed up at each level.
- **Horizon effect:** a fixed look-ahead can miss disaster just beyond the horizon; deeper search from promising states helps but never removes it.

Alpha-beta rules

- α (at MAX nodes) can never decrease; β (at MIN nodes) can never increase. Search depth-first, left to right.
- **Rule 1:** stop below a MIN node whose β ≤ the α of any MAX ancestor (an α cut-off).
- **Rule 2:** stop below a MAX node whose α ≥ the β of any MIN ancestor (a β cut-off).
- Same value as minimax with fewer nodes; with good ordering it can double the searchable depth; worst case it examines as much as minimax. Left-to-right and right-to-left orders give different prunings.

Method for the exam: (1) go down the left branch to the first leaves; (2) back values up, writing α/β at each node; (3) before exploring a sibling, compare with the ancestor bound and draw a cut line when a rule applies; (4) at the root write the final value and mark the winning path.

4.4 Complexity

- B = average branching factor; states at depth L = B^L; total generated $\mathbf{T = B + B^2 + \ldots + B^L = B(B^L - 1)/(B - 1)}$.
- 8-puzzle branching factor is about 2.67, or 1.67 if the move back to the parent is removed. Beam search keeps only the best few states in open. NP-hard problems need heuristics. IDA* combines iterative deepening with A* for linear space.

Exam focus (Dr Baloyi / SCOA032 style)

- Give the two situations for heuristics and explain fallibility.
- Perform alpha-beta on a 4-ply tree and identify the winning path.
- Trace best-first search with open and closed; compute f = g + h; define admissible, monotone and more informed.
- Explain hill-climbing local maxima, the horizon effect and beam search; fill an edit-distance table.

Chapter 5 - Stochastic Methods (self-study chapter)

Study priority: low

- The module outline marks this as a recommended self-study chapter and (per the earlier analysis) no probability questions appeared in 2024-2026. Learn only the points below; the long counting, joint-table and n-gram material from the first version is dropped.

Five lines to know

- Probabilistic reasoning suits problems with no deterministic cause-effect link (diagnosis, language).
- Probability $p(E) = |E| / |S|$; $p(\text{not } E) = 1 - p(E)$; A and B are independent iff $p(A \cap B) = p(A)\, p(B)$.
- Conditional probability: $p(d \mid s) = p(d \cap s) / p(s)$.
- **Bayes' rule:** $p(h_i \mid E) = p(E \mid h_i)\, p(h_i) / \Sigma_k\, p(E \mid h_k)\, p(h_k)$: it turns hard-to-measure posteriors into easier likelihoods.
- **Naive Bayes** assumes the evidence is independent given the hypothesis: $p(E \mid h_i) \approx \Pi_j\, p(e_j \mid h_i)$ (the product over all pieces of evidence, which the first version omitted).

Worked example (car dealers): $p(d_1)=0.2$, $p(d_2)=0.4$, $p(d_3)=0.4$; $p(a_1|d_1)=0.2, 0.4, 0.3$. Then $p(d_2|a_1) = 0.16 / (0.04 + 0.16 + 0.12) = 0.5$.

Binomial: $p(r \text{ successes in } n \text{ trials}) = {}_nC_r \cdot p^r \cdot (1 - p)^{n-r}$. Permutations ${}_nP_r = n!/(n-r)!$; combinations ${}_nC_r = n!/[(n-r)!\, r!]$.

Chapter 6 - Control Algorithms and Production Systems

Study priority: highest

- Per the earlier analysis this is the most heavily examined chapter in 2024 and 2025. Be able to define, trace and explain every box below.

6.1 Recursion

Recursion in computer science

- A recursive procedure has two main components: (1) a **base case** (terminating condition) solved directly without recursion; (2) a **recursive step** that reduces the problem towards the base case by calling itself on a smaller instance.
- Mathematical recursion defines a function in terms of itself; in CS the recursion also needs a stack of activation records, and without a base case it never stops.
- State space search is recursive: move to a child and recurse; if it fails return to the parent and try the next sibling (backtracking). The call stack replaces the explicit open list.

```
function depthsearch(current_state);   % closed is global; open is implicit in the stack
  if current_state is a goal then return SUCCESS;
  add current_state to closed;
  while current_state has unexamined children
    child := next unexamined child;
    if child not member of closed then
      if depthsearch(child) = SUCCESS then return SUCCESS
  return FAIL
```

Pattern-directed search (basis of Prolog): the query is the goal; unification selects rules whose conclusion matches; the premise becomes the subgoal. Conjunctions need consistent bindings (p(X) ∧ q(X) uses the same X); negation as failure: ¬p succeeds only if p fails; rules and facts are tried in the order written.

6.2 Production systems

Architecture and roles

- **Production rules:** condition-action pairs, each a chunk of knowledge (long-term memory).
- **Working memory:** the current state of the problem (short-term memory); it changes as rules fire.
- **Recognise-act cycle (control):** match working memory against rule conditions → conflict set of enabled rules → conflict resolution picks one → fire it → repeat; stop when no rule matches.
- **Condition (left side):** a pattern matched against working memory. **Action (right side):** modifies working memory by adding, deleting or modifying elements (or performing an operation).
- The pure model has no recovery from dead ends; practical systems add backtracking. Shells: OPS5 (XCON), CLIPS (NASA), JESS (Java).

Conflict resolution strategies

- **Refraction:** once a rule has fired on a particular set of working-memory elements it may not fire again on the same data until those elements change. Prevents looping.
- **Recency:** prefer rules that match the most recently added working-memory elements; focuses on one line of reasoning.
- **Specificity:** prefer the more specific rule (more conditions on the left side) over a more general one.
- **First match:** the simplest regime: take the first matching rule in order.

Advantage	What it means (match the statement to the advantage)
Separation of knowledge and control	Control model is independent of the knowledge representation; edit rules without changing control code.
Modularity of rules	Rules can be added, deleted or modified independently; they interact only through working memory.
Tracing and explanation	Each rule is a meaningful chunk; the chain of fired rules shows the line of reasoning.
Natural mapping to state space search	Working-memory states = nodes; rules = transitions; conflict resolution = branch choice.
Pattern-directed control	Rules fire in any order; the current state decides the path.
Heuristic control	Rule order, rule structure and conflict resolution encode heuristics.
Models human problem solving	Newell and Simon: rules as long-term skills, working memory as short-term memory.

6.3 Worked traces

Sorting a string with rules 1: ba → ab; 2: ca → ac; 3: cb → bc, start cbaca, taking the lowest-numbered rule in the conflict set (corrected trace):

Iter	Working memory	Conflict set	Rule fired
0	cbaca	1, 2, 3	1
1	cabca	2	2
2	acbca	2, 3	2
3	acbac	1, 3	1
4	acabc	2	2
5	aacbc	3	3
6	aabcc	none	halt

Forward chaining (data-driven) with rules 1: p∧q → goal; 2: r∧s → p; 3: w∧r → q; 4: t∧u → q; 5: v → s; 6: start → v∧r∧q. Working memory starts with *start*:

Iter	Working memory	Conflict set	Rule fired
0	start	6	6
1	start, v, r, q	6, 5	5
2	start, v, r, q, s	6, 5, 2	2
3	start, v, r, q, s, p	6, 5, 2, 1	1
4	… , goal	6, 5, 2, 1	halt

Diagram to draw: start → (v, r, q); v → s; r ∧ s → p; p ∧ q → goal. Rules fired in order 6, 5, 2, 1. **Goal-driven** on the same rules starts with *goal* in working memory, matches rule **conclusions**, and adds **conditions** as subgoals: rules fired 1, 2, 3, 4, 5, 6.

	Data-driven	Goal-driven
Working memory starts with	Facts	The goal
Matches against	Rule conditions	Rule actions (conclusions)
Firing	Adds the conclusion to working memory	Adds the conditions as subgoals
Ends when	Goal derived	Subgoals reduce to known facts or user answers

6.4 The 3 × 3 knight's tour

Squares are numbered 1-9 row by row. Moves: move(1,6), move(1,8), move(2,7), move(2,9), move(3,4), move(3,8), move(4,3), move(4,9), move(6,1), move(6,7), move(7,2), move(7,6), move(8,1), move(8,3), move(9,2), move(9,4) (16 facts). Recursive production system:

```
∀X path(X, X)
∀X,Y [path(X, Y) ← ∃Z [move(X, Z) ∧ path(Z, Y)]]

Loop-safe version (visited squares recorded in working memory):
∀X,Y [path(X, Y) ← ∃Z [move(X, Z) ∧ ¬(been(Z)) ∧ assert(been(Z)) ∧ path(Z, Y)]]
```

Trace for path(1,2): 1 → 8 → 3 → 4 → 9 → 2. Without the *been* marker the search can loop (move(2,9) then move(9,2)); the fix records visited squares in **working memory**, not a global closed list. assert(X) adds X to working memory and always succeeds. For the full 8 × 8 board replace the move facts by rules such as 'row < 6 ∧ column < 7 → new row = row + 2, new column = column + 1'.

6.5 Blackboard architecture (brief)

Production memory is split into independent, asynchronous, data-driven **knowledge sources** that cooperate through a global **blackboard**. HEARSAY-II (speech understanding) uses levels from waveform to phrases with a scheduler; HEARSAY-III generalises it and moves the scheduler to a second blackboard.

Exam focus (Dr Baloyi / SCOA032 style)

- Define a production system and its three parts; explain the roles of condition, action, working memory and the recognise-act cycle.
- Define and contrast refraction, recency and specificity; match a statement to an advantage of production systems.
- Explain the two components of recursion (base case, recursive step) and trace the recursive search.
- Draw the forward-chaining rule diagram and trace working memory; trace the 3 × 3 knight's tour (path(1,2)); trace the sorting production system.

Chapter 7 - Knowledge Representation

7.1 Three approaches and semantic networks

	Weak method	Strong method	Agent-based
Idea	General heuristics, domain independent	Large amounts of explicit domain knowledge	No central knowledge base; situated, autonomous agents
Examples	Logic Theorist, GPS (means-ends analysis)	Expert systems	Brooks' subsumption, robot soccer
Weakness	No single heuristic works everywhere	Acquiring and organising knowledge	Coordination and scale

- **Why not just logic?** ∀X (cardinal(X) → red(X)) is equivalent to ∀X (¬red(X) → ¬cardinal(X)), so a white sheet of paper would count as evidence that cardinals are red. English implies a correlation.
- **Associationism:** meaning is a network of associations. **Semantic network:** nodes = concepts, labelled arcs = relations, supports **inheritance** (Collins and Quillian: properties stored at the highest level; exceptions at the most specific).
- **Case frames:** a verb node with case links (agent, object, instrument, location, time). **Conceptual dependency** (Schank): primitive ACTs such as ATRANS (give), PTRANS (go), MTRANS (tell), INGEST (eat); sentences with the same meaning get the same graph (canonical form). Criticisms: low-level primitives, canonical reduction not reliable.

7.2 Scripts

Script (Schank and Abelson)

- A structured representation of a **stereotyped sequence of events** in a context, used to fill in missing information in language understanding. Five components: entry conditions, props, roles, scenes, results.

Component	Restaurant script (design this in the exam)
Name / track	Restaurant (for example 'Mpfuxeto Baloyi's restaurant').
Entry conditions	Customer is hungry and has money; the restaurant is open.
Props	Tables, chairs, menu, food, bill, money, tip.
Roles	Customer (diner), waiter (server), chef (cook), cashier/owner.
Scene 1: Entering	Customer enters, looks for a table, is seated by the waiter.
Scene 2: Ordering	Customer reads the menu, calls the waiter, orders; waiter tells the chef.
Scene 3: Eating	Chef prepares food, waiter serves, customer eats.
Scene 4: Paying and exiting	Customer asks for the bill, pays (and tips), leaves.
Results	Customer is full and satisfied and has less money; owner has more money; restaurant has less food.

Have a backup script ready (clinic visit or supermarket) with the same five components. **Problems:** script *match* (choosing the right script is hard) and *between-the-lines* (unexpected events break the script). **MOPs** are smaller components combined dynamically.

7.3 Frames

Frame (Minsky)

- A remembered framework for a **stereotyped situation** with named **slots** and values, adapted to the situation by changing details (hotel room, hotel chair, hotel phone).
- **Class inheritance (specialisation) slots** (is-a, ako, instance-of): link a frame to its superclass so it inherits slots and defaults, for example hotel phone → phone → communication device.
- **Procedural attachment slots (demons):** attach code to a slot: *if-needed / when-needed* computes a value on demand; *if-added / when-changed* triggers side effects such as consistency checks when the value is set or changed.
- Other slot types: defaults (chairs have four legs), descriptors of requirements, new-instance information (left open until needed). Default reasoning is non-monotonic. Frames led to object-oriented programming.

7.4 Conceptual graphs (Sowa)

Conceptual graph

- A finite, connected, **bipartite** graph: nodes are **concepts** (boxes) or **conceptual relations** (ovals); **arcs (links)** join concepts only to relations and relations only to concepts. Each graph is one proposition; a relation of arity n has n arcs.

- Label = type : referent, for example [dog : emma]. A marker #n is a unique individual; * is a generic marker.
- **Type hierarchy:** a partial order forming a lattice with universal type T and absurd type ⊥.

Experiencer relation vs experiencer link

- The **experiencer relation** (EXPR) is a conceptual relation *node* (an oval) naming the semantic role of a conscious entity that experiences a state, e.g. [PERSON: Tom] ← (EXPR) ← [LIKE] → (OBJ) → [PIZZA].
- A **link** is the directed arc (arrow) that connects a concept box to a relation oval; links carry no meaning of their own beyond direction and which node they join. In a plain semantic network the relation is just the label on an arc, whereas conceptual graphs make the relation a separate node connected by links.

Rule	Effect
Copy	Exact copy of a graph.
Restrict	Replace a generic marker by an individual, or a type by a subtype.
Join	Combine two graphs on an identical concept node.
Simplify	Delete a duplicate relation.

Translation to predicate calculus: [dog : emma] with colour brown becomes ∃X1 (dog(emma) ∧ color(emma, X1) ∧ brown(X1)). Conceptual graphs are as powerful as predicate calculus; join and restrict implement inheritance and are meaning-preserving, not truth-preserving.

7.5 Brief notes

- **Subsumption (Brooks):** layered asynchronous finite state machines, no central model; higher layers suppress lower ones (AVOID, WANDER, EXPLORE).
- **Copycat:** workspace, slipnet, coderack, temperature.
- **Multi-agent criteria:** situated, autonomous, flexible (responsive and proactive), social.

Exam focus (Dr Baloyi / SCOA032 style)

- Design a restaurant script with all five components (and a second script on demand).
- Name and explain frame slot types: inheritance and procedural attachment (demons).
- Distinguish the experiencer relation from links in a conceptual graph; draw and translate a conceptual graph.
- Describe semantic networks, inheritance, case frames and conceptual dependency.

Chapter 8 - Strong Method Problem Solving

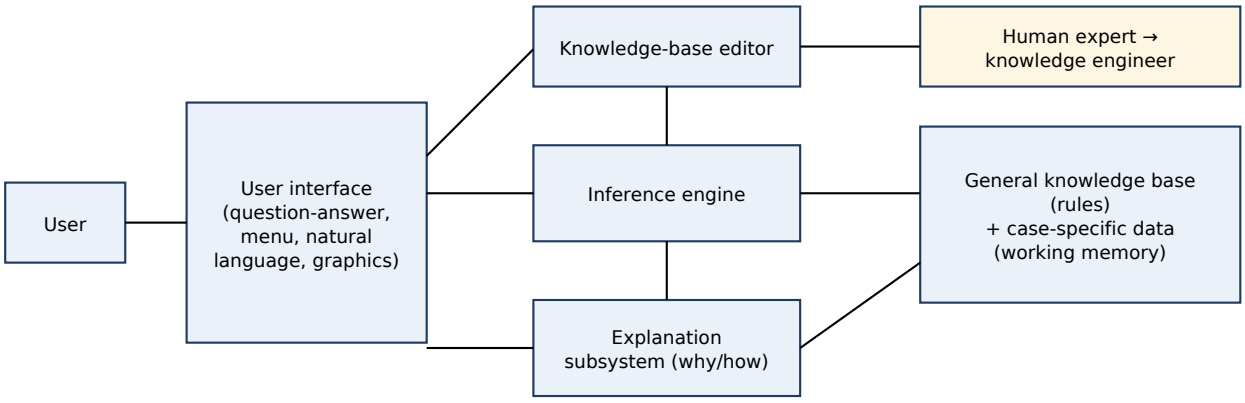
'The power resides in the knowledge' (Feigenbaum): expert systems are knowledge-rich even if methods-poor.

8.1 Is an expert system appropriate? (Luger's guidelines)

Seven guidelines (be ready to give any six)

1. The need for the solution justifies the cost and effort of building an expert system.
2. Human expertise is not available in all the situations where it is needed (scarce, retiring, remote).
3. The problem can be solved using symbolic reasoning (not purely numerical, no dexterity or perception).
4. The problem domain is well structured and does not require commonsense reasoning.
5. The problem cannot be solved easily by traditional computing methods.
6. Cooperative and articulate human experts exist.
7. The problem is of proper size and scope (narrow and manageable).

8.2 Architecture of a typical expert system



Module	Role
User interface	Hides complexity; question-and-answer, menu, natural language or graphics.
Knowledge base	Domain knowledge, usually if-then rules.
Case-specific data (working memory)	Facts, partial conclusions and certainty measures for the current case.
Inference engine	Interpreter that runs the recognise-act cycle (forward or backward chaining).
Explanation subsystem	Answers **why** (why do you ask?: the current rule) and **how** (how did you conclude this?: the trace of rules).
Knowledge-base editor	Lets the knowledge engineer add and check rules; consistency checks.
Shell	All modules with an empty knowledge base (CLIPS, JESS).

- **Roles:** the **domain expert** supplies knowledge; the **knowledge engineer** elicits and encodes it; the **end user** sets interface and explanation needs.
- **Why separate knowledge from the inference engine:** knowledge in natural if-then form, changes without side effects, reusable control and interface software.
- Development is **exploratory programming**: build a prototype, test with the expert, extend; the system is never 'finished'.

8.3 Rule-based reasoning (car example)

Rules: (1) engine getting gas ∧ engine turns over → spark plugs; (2) engine does not turn over ∧ lights do not come on → battery or cables; (3) engine does not turn over ∧ lights come on → starter motor; (4) gas in tank ∧ gas in carburetor → engine getting gas.

- **Goal-driven:** put 'the problem is X' in working memory; rule 1 matches (an or choice) and its premises become and subgoals; subgoals matching no rule conclusion are put to the user.
- **Data-driven:** compare working memory with rule conditions, add conclusions, repeat; opportunistic but less focused, with weaker explanations.
- Control: put the premise most likely to fail or cheapest to test first.

8.4 Other reasoning styles (one-liners)

Approach	Idea	Main weakness
Rule-based	Experiential if-then rules	Brittle at the edges of the domain; no deep knowledge
Model-based	Simulate the structure and function of a device; a fault is a discrepancy	Needs an explicit model; complex
Case-based	Retrieve, modify, apply, save past cases	Weak explanation; indexing is hand-crafted

8.5 Planning and STRIPS

- A planner finds a sequence of actions that achieves a goal; planning is state space search over world states.
- **Frame problem:** specifying what an action changes and what stays the same. **STRIPS** avoids frame axioms: each operator has **P** (preconditions), **A** (add list) and **D** (delete list); everything not mentioned stays true.
- Subgoals that conflict (for example on(b,a) ∧ on(a,c)) mean linear planning fails. Triangle tables save generalised macro operators.

Operator	Preconditions P	Add list A	Delete list D
pickup(X)	gripping( ) ∧ clear(X) ∧ ontable(X)	gripping(X)	ontable(X) ∧ gripping( )
putdown(X)	gripping(X)	ontable(X) ∧ gripping( ) ∧ clear(X)	gripping(X)
stack(X,Y)	clear(Y) ∧ gripping(X)	on(X,Y) ∧ gripping( ) ∧ clear(X)	clear(Y) ∧ gripping(X)
unstack(X,Y)	clear(X) ∧ gripping( ) ∧ on(X,Y)	gripping(X) ∧ clear(Y)	gripping( ) ∧ on(X,Y)

Triangle table: actions on the diagonal; each action's preconditions in the row before it, its add/delete effects in the column below; the leftmost column gives the preconditions of the whole macro, the bottom row what it achieves. The *n*th kernel is the set of predicates that must hold before action n; execute the highest-numbered action whose kernel is true.

Exam focus (Dr Baloyi / SCOA032 style)

- Give six (or seven) guidelines for when an expert system is appropriate.
- Draw the typical expert-system architecture and explain each module, why/how and the three human roles.
- Trace goal-driven and data-driven rule runs; compare rule-based, model-based and case-based reasoning.
- Write STRIPS operators (P, A, D) and explain the frame problem and triangle tables.

Appendix A - Chapters 10, 14 and 15

Why this appendix

- The module outline reportedly includes Chapters 10, 14 and 15 for late-semester assessment. This is a lean summary of the usual exam items; check the outline for the exact sections.

A.1 Chapter 10 - Symbol-based machine learning

Definitions

- **Machine learning:** a program improves its performance on a task through experience.
- **Inductive learning:** generalising from specific examples to a general rule or concept (the result may be wrong, so it is not truth-preserving).
- **Concept learning:** learning a general description of a class from positive examples and negative examples (near misses).

Version space search (candidate elimination)

- Keeps two boundary sets: **S** (most specific hypotheses consistent with the data) and **G** (most general hypotheses).
- A **positive** example generalises S (and removes members of G that do not cover it); a **negative** example specialises G (and removes members of S that cover it).
- The concept is learned when S and G converge to the same description.

ID3 decision tree induction

- Builds a tree top-down: at each node choose the attribute with the highest **information gain**, split the examples, and recurse until each leaf is pure.
- Entropy: $\mathbf{H(S) = \Sigma - p_i \log_2 p_i}$, where p_i is the proportion of class i. Gain(S, A) = H(S) - Σ $(|S_v|/|S|)$ $H(S_v)$ over the values v of attribute A.
- Example: 9 positive and 5 negative examples give $H = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.940$.

A.2 Chapter 14 - Automated reasoning

Resolution refutation: the recipe

1. Translate the English statements into predicate calculus.
2. Convert to **clause form**: eliminate →; move ¬ inward (De Morgan, quantifier rules); standardise variables apart; Skolemize ∃; drop ∀; distribute ∨ over ∧; split into clauses.
3. **Negate the goal** and add it to the clause set.
4. Resolve pairs of clauses (unifying complementary literals) until the **empty clause** [ ] is derived. Deriving [ ] shows the negated goal is inconsistent, so the goal is proved.

Worked example. Everyone who studies passes; everyone who passes is happy; Thandi studies. Prove happy(thandi).

```
1. ¬studies(X) ∨ passes(X)
2. ¬passes(Y) ∨ happy(Y)
3. studies(thandi)
4. ¬happy(thandi)                 (negated goal)
5. ¬passes(thandi)               resolve 4 with 2, {thandi/Y}
6. ¬studies(thandi)              resolve 5 with 1, {thandi/X}
7. [ ]                           resolve 6 with 3   → goal proved
```

GPS and means-ends analysis: compare the current state with the goal, find the differences, and use a **difference table** to pick an operator that reduces the largest difference; apply it (setting up its preconditions as subgoals) and repeat.

A.3 Chapter 15 - Understanding natural language

Level of knowledge	One-line definition
Prosody	Rhythm and intonation of speech.
Phonology	The sounds that make up words.
Morphology	The structure of words (roots, prefixes, suffixes).
Syntax	How words combine into legal sentences (grammar).
Semantics	The meaning of words and sentences.

Level of knowledge	One-line definition
Pragmatics	How language is used in context to achieve goals.
World knowledge	General knowledge of the world needed to understand language.

Context-free grammar (rewrite rules): sentence → np vp; np → n | art n; vp → v | v np; art → a | the; n → man | dog; v → likes | bites. A **parse tree** shows how a sentence is derived: top-down starts at *sentence* and expands; bottom-up starts from the words and reduces. Parse of *the dog bites the man*:

```
sentence
├── np
│   ├── art → the
│   └── n   → dog
└── vp
    ├── v   → bites
    └── np
        ├── art → the
        └── n   → man
```

The grammar checks syntax only, so 'the man bites the dog' is also accepted. For *the dog likes men* you would need 'men' in the grammar (for example n → men with np → n).

Exam focus (Dr Baloyi / SCOA032 style)

- Define machine learning, inductive learning and concept learning; run version space search and compute entropy / information gain.
- Run a resolution refutation from English statements to the empty clause.
- List and define the seven levels of language knowledge; draw a parse tree from given grammar rules.

Cheat sheet - formulas, algorithms and comparisons

Algorithm	open is a…	Shortest path?	Space	Notes
Backtrack	NSL + SL + DE	No	Linear	DFS with a dead-end list
Depth-first	Stack	No	Linear	Use a depth bound
Breadth-first	Queue	Yes	Exponential	Level by level
Iterative deepening	DFS to bound 1, 2, 3…	Yes	Linear	Time $O(B^n)$
Hill climbing	None	No	Constant	Local maxima
Best-first	Priority queue	No	Exponential	Recovers via open
A*	Priority queue by f	Yes if $h \leq h^*$	Exponential	$f = g + h$
Alpha-beta	Depth-first to ply n	n/a	Linear	Same value as minimax

Topic	Formula
Evaluation function	$f(n) = g(n) + h(n)$; A*: $h(n) \leq h^*(n)$
Monotone heuristic (corrected)	$h(n_i) - h(n_j) \leq cost(n_i, n_j)$ for any descendant n_j of n_i; $h(goal) = 0$
More informed	$h_1(n) \leq h_2(n)$ for all $n \Rightarrow h_2$ more informed
States generated	$T = B + B^2 + \ldots + B^L = B(B^L - 1)/(B - 1)$
Search space sizes	Tic-tac-toe 9! = 362,880; TSP (N-1)!; chess about 10^{120}; checkers about 10^{40}
Bayes	$p(h_i\|E) = p(E\|h_i)\, p(h_i) / \Sigma_k\, p(E\|h_k)\, p(h_k)$
Naive Bayes (corrected)	$p(E\|h_i) \approx \Pi_j\, p(e_j\|h_i)$
Binomial (corrected)	${}_nC_r \cdot p^r \cdot (1 - p)^{n-r}$
Edit distance	insert 1, delete 1, replace 2, match 0
Entropy / gain	$H(S) = \Sigma\, -p_i \log_2 p_i$; Gain $= H(S) - \Sigma\, (\|S_v\|/\|S\|)\, H(S_v)$
Euler path	Exists iff the number of odd-degree nodes is 0 or 2

Scheme	Key idea
Predicate calculus	Sound, declarative; not decidable
Semantic network	Graph of concepts; inheritance
Script	Entry conditions, props, roles, scenes, results
Frame	Slots, defaults, inheritance, demons (if-needed, if-added)
Conceptual graph	Bipartite concepts and relations; copy, restrict, join, simplify
Production system	Rules + working memory + recognise-act cycle; conflict resolution
Expert system	Knowledge base + inference engine + explanation (why/how) + editor + user interface